CareerVault: A Dual-Approval, Merkle-Anchored System for Verifiable Employment Credentials

[Author Name 1]
Dept. of [Department]
K. J. Somaiya School of Engineering
Mumbai, India
[email1@domain]

[Author Name 2]
Dept. of [Department]
K. J. Somaiya School of Engineering
Mumbai, India
[email2@domain]

[Author Name 3]
Dept. of [Department]
K. J. Somaiya School of Engineering
Mumbai, India
[email3@domain]

[Author Name 4]
Dept. of [Department]
K. J. Somaiya School of Engineering
Mumbai, India
[email4@domain]

***Abstract*—Verifying an applicant's employment history is slow, manual and repeated by every employer. This paper presents CareerVault, a working system in which an organisation issues employment documents (experience and relieving letters, salary certificates, letters of recommendation) that are approved by both a manager and an HR officer, held by the employee in a wallet, and verifiable by anyone. Document content is canonicalised (RFC 8785), salted and hashed with SHA-256; the manager and the HR officer each sign a distinct, role-bound statement with the organisation's RSA-2048 key; and a daily job folds all newly issued hashes into a Merkle tree whose 32-byte root is anchored in a Solidity registry deployed on the Polygon Amoy testnet. A downloadable credential file lets a verifier recheck the hash, both signatures, Merkle inclusion and the on-chain root without contacting the server. Sensitive fields are sealed with AES-256-GCM envelope encryption, and erasure deletes the per-document salt so that the anchored hash can no longer be tied to content. A consent-gated talent-matching module retrieves candidates with pgvector and explains scores with TreeSHAP. We describe the design, a threat analysis and measured costs: a Merkle proof for a batch of 100,000 documents is 544 bytes and verifies in about 45 µs. We also state plainly what is not yet real: one custodial key per organisation, mocked payments and e-mail, and a ranker that has not been trained on outcome data.**

***Index Terms*—credential verification, blockchain anchoring, Merkle tree, digital signatures, verifiable credentials, envelope encryption, right to erasure, explainable AI, TreeSHAP**

I. Introduction

Employers routinely act on documents they cannot cheaply authenticate. Background-verification vendors report non-trivial discrepancy rates: AuthBridge's 2025 report, covering checks run between October 2024 and March 2025, puts the rate at 6% for white-collar candidates and 9.46% in IT/ITES, with inflated titles, exaggerated tenures and unverifiable employers among the commonest flags [1], and an earlier HireRight study of 218,427 checks in India found discrepancies in 23% [2]. Both figures are vendor-published and self-reported and are not comparable with each other, but they show why verification is a standing cost of hiring. Verification today usually starts after an offer and proceeds by e-mail, telephone or a paid agency, which delays onboarding and is unaffordable for many small employers [3].

Anchoring cryptographic fingerprints of documents on a public ledger is an established answer to tamper evidence [4], [5], [6]. Systems for academic credentials [7], [8], [9] and for hiring documents [3], [10] have applied it. Four questions remain less well covered in the literature we reviewed. Who exactly attested to a document, and can one person alone produce that attestation? How can content stay private and erasable when its fingerprint sits on an immutable ledger? Can a third party verify a document if the operator disappears? And what does an end-to-end, deployable system look like, beyond a prototype that stores hashes?

We present CareerVault, a system built to answer these questions for employment documents in an Indian context, and make the following contributions:

- **Role-bound dual approval.** A manager and an HR officer each sign a distinct statement that binds the document hash to a role and a membership identifier, so the two signatures are not interchangeable (Section V-C).
- **Salted hash, Merkle batching and an offline credential.** Documents are hashed with a per-document salt, batched daily into a Merkle tree, and anchored by a 32-byte root; a self-contained credential file lets anyone recheck every step without the operator (Sections V-D to V-F).
- **Layered data protection.** Application-level envelope encryption with fail-closed reads and a batch integrity gate, tiered disclosure to anonymous verifiers, and erasure by salt deletion (Section VII).
- **Consent-gated, explainable matching.** A pipeline from consented skill extraction to vector retrieval and TreeSHAP attributions, with an explicit statement of what the ranker does and does not learn (Section VIII).
- **Implementation, measurements and limits.** A deployed implementation, micro-benchmarks of its cryptographic primitives, a threat analysis, and a candid list of limitations (Sections IX and X).

We followed the design-science research methodology [11]: the problem and objectives (Section III) precede the artefact (Sections IV–VIII), which we demonstrate through a running deployment and evaluate through measurements and a threat analysis (Section IX). Section II reviews related work, Section XI lists planned work taken from the project's own roadmap, and Section XII concludes.

II. Background and Related Work

A. Hash Anchoring and Merkle Batching

Haber and Stornetta proposed time-stamping a document by publishing a hash of it [4]; Merkle trees [5] let many hashes

share one published value. OpenTimestamps aggregates many digests into one Bitcoin transaction and returns a proof that is completed once the anchor confirms [6], the same incomplete-then-complete pattern that CareerVault exposes as a *pending-anchor* verdict. Certificate Transparency prefixes leaf and interior hashes with different bytes to separate their domains [12]; we return to this in Section X.

B. Blockchain-Based Credential Systems

Table I summarises the systems we compare against. Marella and Vijayan store, on a Hyperledger Fabric consortium ledger, the hash of a document and the hash of an identifier (social-security number plus last name); a hiring manager uploads the document to a web application that recomputes and compares the hash [3]. Fabric is a permissioned platform with pluggable consensus [13]. MIT's 2017 pilot gave 111 students digital diplomas whose hash was written to the Bitcoin blockchain [7], using the Blockcerts format. Baldi *et al.* analysed Blockcerts and showed that its issuer-authentication step can be defeated by fabricated issuer profiles [14], a weakness that bears directly on how a verifier binds a signing key to an organisation (Section V-F). EduCTX represents higher-education credits as tokens on an Ark-based platform [8]; Gräther *et al.* describe an Ethereum platform that stores certificate fingerprints and supports certifiers, learners and employers [9]; and Herbke *et al.* manage the lifecycle of résumés with decentralised identifiers and verifiable credentials on EBSI [10].

TABLE I
COMPARISON WITH RELATED SYSTEMS

Work	Ledger	On-chain payload	Focus
Marella & Vijayan [3]	Hyperledger Fabric (consortium)	Hash of document and of identifier	Hiring documents from several issuer types
Blockcerts / MIT [7], [14]	Bitcoin (MIT pilot)	Certificate hash; Merkle batching	Academic credentials
EduCTX [8]	Ark-based platform	ECTS credit tokens	Higher-education credit
Gräther *et al.* [9]	Ethereum	Certificate fingerprints	Issue, validate, share
Herbke *et al.* [10]	EBSI	DID/VC infrastructure	Résumé lifecycle
This work	Polygon Amoy (public, EVM)	Daily Merkle root; revocation flag	Employment documents; dual approval; offline proof; matching

C. Verifiable Credentials

The W3C Verifiable Credentials Data Model 2.0 defines a JSON-LD structure for signed claims [15]. CareerVault's credential file is VC-shaped: it declares the W3C v1 context beside a project-specific context and carries a project-specific proof type. We have not tested it against the W3C conformance suite and do not claim conformance.

D. Explainable Ranking and Retrieval

SHAP assigns each feature an additive contribution to a single prediction [16], and TreeSHAP computes these exactly in polynomial time for tree ensembles [17]. LightGBM [18] exposes TreeSHAP natively. Sentence embeddings [19], here from the compact MiniLM family [20], support semantic retrieval, and the pgvector extension [21] provides exact and HNSW-based [22] nearest-neighbour search inside PostgreSQL. Annex III of the EU AI Act lists systems used to recruit or select natural persons among its high-risk use cases [23], which motivates explainability and consent in a matching feature.

III. REQUIREMENTS AND THREAT MODEL

The actors are the *holder* (any authenticated user), the organisation's *manager*, *HR officer* and *administrator*, a *recruiter*, a *verifier* (anonymous, or holding an API key), and the *platform operator*. We derived the following requirements.

- **R1 Issuer authenticity:** a document must trace to a key that belongs to a domain-verified organisation.
- **R2 Separation of duties:** no single member should be able to issue a document alone, except through one explicitly documented bulk path.
- **R3 Tamper evidence:** any change to content after issuance must be detectable by a party that holds only the credential.
- **R4 Operator independence:** verification must remain possible, offline, if the operator is unavailable.
- **R5 Privacy:** anonymous verifiers see a minimum; data is encrypted at rest; a holder can erase their data.
- **R6 Revocation and expiry:** an issuer can withdraw a document, and validity can lapse.
- **R7 Consent and explanation:** document content is used for matching only with consent, and every score is explained.
- **R8 Low marginal cost:** anchoring cost must not grow with the number of documents.

We consider six adversaries: A1, a dishonest holder who alters or forges a document; A2, a reader of database dumps or backups; A3, a database *writer* without any application keys; A4, a compromised application server that holds the master key; A5, a malicious or colluding issuer insider; and A6, network faults between the verifier and the chain. Out of scope are the truthfulness of what an honest-looking issuer attests, compromise of a holder's own device, and attacks on Polygon consensus.

IV. SYSTEM ARCHITECTURE

CareerVault is a monorepo of six packages (Fig. 1): a React 19 / Vite 7 / Tailwind 4 single-page application with five role portals and a public verifier; a NestJS 11 API with 77 REST endpoints over PostgreSQL through Prisma 7, with the pgvector extension; a FastAPI service for skill extraction, embedding and ranking; a Solidity 0.8.24 contract built with Hardhat; Playwright end-to-end tests; and a standalone Node verifier. The code base is about 37,000 lines across TypeScript, Python and Solidity, and the Git history spans 143 commits from March to October 2026. The relational model has 21 tables.

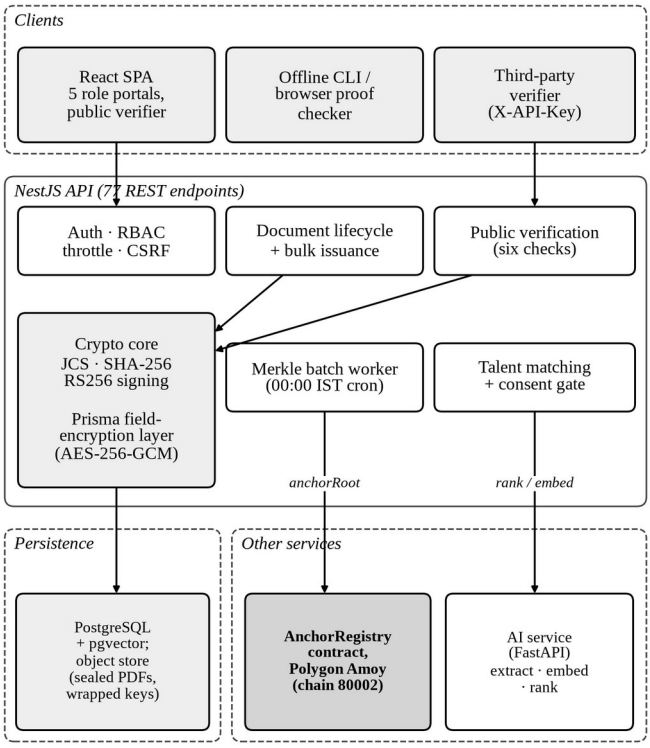


Fig. 1. System architecture. Both the verification module and the batch worker call the registry; the offline checker reads it directly.

Adapters. Every heavy integration sits behind an interface with interchangeable drivers: key management, blockchain, payments, e-mail, object storage and DNS. Table II lists the implemented drivers and the values used in the deployed demonstration. All drivers except the blockchain one are local or mock implementations in that deployment, and the server logs a warning at boot for each.

TABLE II
ADAPTER DRIVERS AND THE DEPLOYED DEMO CONFIGURATION

Concern	Implemented drivers	Demo deploy
Key management	local disk; Supabase bucket	Supabase bucket
Blockchain	local JSON ledger; Polygon Amoy	Polygon Amoy
Domain check	local (always passes); real DNS TXT	local (bypass)
Payments	mock only	mock
E-mail	console; Gmail SMTP	console
File storage	local disk; Supabase bucket	Supabase bucket

Identity and access. Passwords are hashed with bcrypt at cost 12 [24]. Access tokens are RS256 JWTs [25] with a configurable lifetime (default one day); refresh tokens are opaque random strings stored only as SHA-256 digests, rotate on every use, and a reuse outside a short grace window revokes the whole token family. Cookie-authenticated refresh and logout routes are protected by a double-submit CSRF token in production, authentication routes are rate-limited per IP and per e-mail, and external signers without accounts use single-use, 15-minute magic links. Authorisation is by membership: every request loads the caller's memberships (administrator, manager, HR, recruiter) and a recruiter has no ambient access to organisation documents.

Organisation onboarding. A founder's e-mail domain must equal the domain claimed. The administrator then proves control of the domain by publishing a TXT record containing a random token. Only after that check does the system generate the organisation's RSA-2048 key pair, store the private key wrapped under a master key, record the public key, and record an identifier of the form `did:careervault:<id>`; no DID resolution method is implemented.

V. Credential Lifecycle and Protocol

A. Lifecycle

A document moves through seven states (Fig. 2). A holder requests a document from an organisation; the server assigns a manager. Only the assigned manager can sign; there is no administrator override, because the signature attests to who vouched for the holder. Signing moves the document to PENDING_HR, where an HR officer either returns it (clearing its hash, salt and manager signature) or approves it. Approval requires an HR membership held by a *different user* than the signer, and the document becomes ISSUED. A daily batch moves it to ANCHORED. An HR officer may revoke an issued or anchored document, and a daily job marks documents past their validity window EXPIRED (90 days for every type except letters of recommendation, which do not expire). All state transitions are written as status-guarded updates inside transactions, so two concurrent signs or approvals cannot both succeed, and every transition snapshots a version row and writes an audit entry.

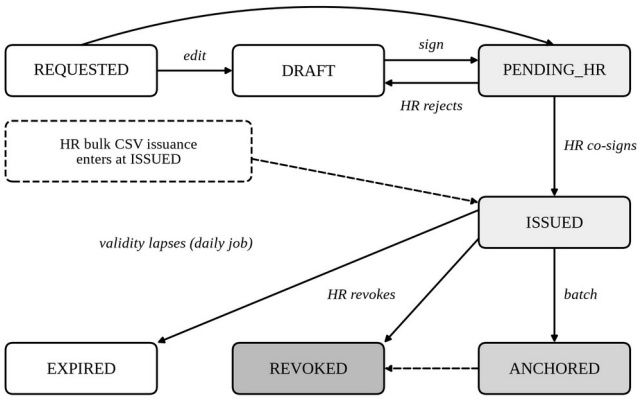


Fig. 2. Document lifecycle. REJECTED is an action returning to DRAFT, not a state; a manager can also return a request to the holder.

B. Content Model

Three document types are supported, each with a server-side schema that whitelists fields and rejects unknown ones: experience/relieving letters, salary certificates and letters of recommendation. The schemas are India-first: salary amounts are integers in paise, with PAN masked and UAN recorded, and the server alone computes gross earnings, total deductions and net pay (net pay must not be negative). Date orderings are checked. The server injects a schema version, the issue date

and a reference number, stamps the signer's name and e-mail from the authenticated session rather than the request body, and drops empty values before hashing so that logically equal subjects serialise identically. The result is the flat *credentialSubject c* that is stored, hashed and signed.

C. Hash and Role-Bound Signatures

At signing, a fresh 32-byte salt is drawn and the document hash is computed over the RFC 8785 canonical form [26] of *c*:

$$h = SHA\text{-}256\left(JCS(c) \parallel salt \right) \quad (1)$$

The salt is appended as 64 hexadecimal characters of UTF-8 text. The manager and the HR officer do not sign *h*. RSASSA-PKCS1-v1_5 signatures [27] are deterministic, so two signatures over the same bare hash by the same key would be byte-identical and would prove nothing about who approved what. Each instead signs a statement that binds the hash to a role *r* and a membership identifier *m*:

$$s_r = SHA\text{-}256\left(JCS(\{v{:}1,\ h,\ r,\ m\}) \right) \quad (2)$$

The signature is then

$$\sigma_r = RS256(sk,\ s_r) \quad (3)$$

where the statement is a canonical JSON object with a version field, the hash, the role *r* (MANAGER at signing, HR at approval) and the membership identifier *m*. Both signatures use the organisation's single RSA-2048 key *sk*. Separation of duties is therefore *enforced* by the application (only the assigned manager signs; the approver must be an HR member and a different user) and *recorded* by the signed statements, but it is not proven by two independent keys; we return to this in Section X. At signing time the organisation's public key is copied onto the document row and verification uses that pinned key, so replacing an organisation key later does not invalidate earlier documents. Approval refuses to co-sign if the key changed after the manager signed. Bulk issuance by HR creates documents directly in ISSUED; both statements then name the same acting HR member, which is honest in the record, and salary certificates are excluded from the bulk path because a single CSV row cannot express their breakdown.

D. Merkle Batching and Anchoring

A worker runs at midnight India time, or on demand, over ISSUED documents that have no proof yet. It first applies an *integrity gate* (Section VII-A): a candidate is included only if its encrypted fields open and its content and salt still recompute its hash; a failing document is skipped and left un-anchored for review. The remaining hashes become leaves of a SHA-256 Merkle tree whose interior nodes hash the bytewise-sorted pair of children:

$$N = SHA\text{-}256\left(min(a,\ b) \parallel max(a,\ b) \right) \quad (4)$$

Sorting makes proofs direction-free; an odd node is promoted rather than duplicated, and a single-leaf root equals the leaf. A proof for *n* leaves contains $\lceil \log_2 n \rceil$ sibling hashes:

$$|\pi| = 32 \lceil \log_2 n \rceil \ bytes \quad (5)$$

The root is anchored with `anchorRoot(root, n)`. The batch asks the contract whether the exact root already exists and only then sends a transaction, so a retry after a partial failure never spends a second transaction; the database write (root row, one proof row per document, status flip) is a single transaction, and a document revoked in the meantime is not flipped back. The chain adapter bounds every RPC call to 10 s, waits for a configurable number of confirmations (default two), serialises writes from one wallet, floors the priority fee, and polls for receipts itself rather than relying on a library subscription that can crash the process on a failed poll. An in-process mutex prevents the cron job and a manual run from overlapping.

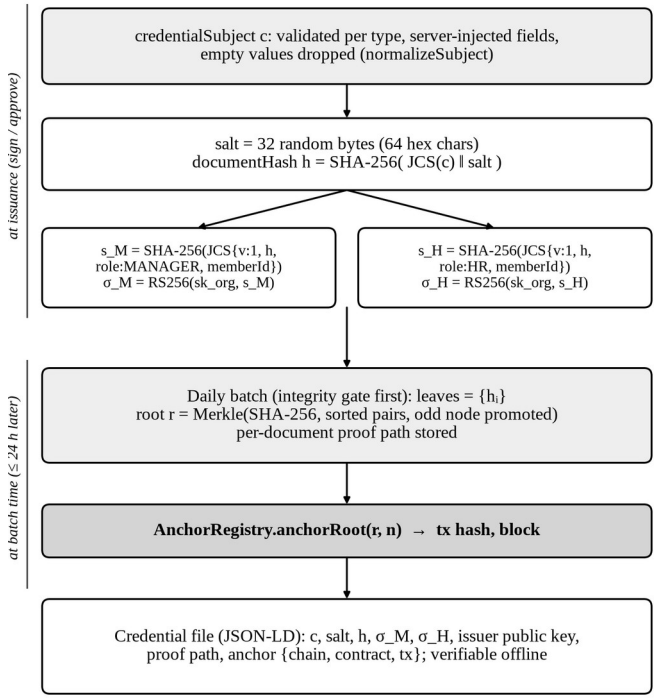


Fig. 3. From content to anchored credential. Signing happens at issuance; the tree is built later, at batch time, so a document is usable before it is anchored.

E. Public Verification

The verification endpoint recomputes every guarantee and trusts no stored validity flag. It returns six checks (Table III) and one of six verdicts. A document that is issued, hash-intact and dual-signed but not yet batched yields VERIFIED_PENDING_ANCHOR, which is a pass state; once its proof reconciles to a root that exists on-chain it becomes VERIFIED. If the chain is unreachable the anchor check degrades to pending, since the proof is still valid locally. Revocation and expiry are decided by the database, with the on-chain flag as a secondary note. The remaining verdicts are REVOKED, EXPIRED, INVALID and NOT_FOUND.

TABLE III
THE SIX VERIFICATION CHECKS

#	Check	Passes when
1	Document on record	status is ISSUED, ANCHORED, REVOKED or EXPIRED
2	Content integrity	SHA-256(JCS(c) ‖ salt) equals the stored hash
3	Issuer signature	RS256 over s_M verifies under the pinned key
4	Approver signature	RS256 over s_H verifies under the same key
5	Blockchain anchor	proof folds to the stored root and the root exists on-chain; pending if unbatched or chain unreachable
6	Revocation status	not revoked and not past expiry

An anonymous lookup by hash discloses only a per-type allow-list of non-sensitive fields: salary amounts, PAN/UAN and contact details are withheld, and for salary certificates so is the holder's name. A holder who creates a share link opts that one document into full disclosure. Links carry a 192-bit random token, an optional expiry and view cap (the cap is claimed atomically so concurrent views cannot exceed it), and are deactivated on revocation. Every public verification writes an audit entry in a compliance tier that the retention job never purges.

F. Offline Credential and Verifiers

The holder can download a credential file containing the subject, salt, hash, both signatures with the member identifiers, the issuer public key, the Merkle proof path, the anchor (chain id, contract address, transaction hash, block) and a textual verification procedure. A standalone Node verifier, which imports nothing from the server, and an equivalent in-browser checker perform five steps: recompute the hash; verify both statements; fold the Merkle proof; check that the contract address is the pinned registry for the chain; and call `verifyRoot` and `isRevoked` and confirm the `RootAnchored` event of the anchoring transaction. The pinned registry for chain 80002 is built into the verifier, and known-answer vectors cover RFC 8785 key ordering (including the UTF-16 code-unit case that naive implementations get wrong), the document hash, the role statements and a Merkle tree.

What a passing verification does *not* prove is that the embedded issuer key belongs to the named organisation: anyone can assemble a credential with their own key and any issuer name. This is the same class of weakness that Baldi *et al.* found in Blockcerts [14]. The verifier therefore prints the key's SHA-256 fingerprint for an out-of-band comparison, and states this limit in its final summary line. The PDF issued to the holder is a human-readable artefact only and is not used for verification.

Verifier API. Third parties can verify up to 100 hashes per request with an `X-API-Key`. Keys are stored as digests, require a live subscription of the matching tier, and are limited to 100 (basic) or 1,000 (enterprise) requests per minute and 1,000 or 50,000 per month, enforced by an in-memory sliding window and a persisted counter. Billing is mocked.

VI. THE ANCHORREGISTRY CONTRACT

`AnchorRegistry` (Solidity 0.8.24, optimiser 200 runs) stores a record for each anchored root (root, document count, timestamp, submitting address) and a revocation timestamp for each revoked document hash. `anchorRoot` reverts on a zero root or one that already exists, so anchors are append-only; `batchAnchorRoots`, `revokeDocument` (idempotent) and `batchRevokeDocuments` complement it. The deployer is the owner and first authorised anchor; the owner can add and remove authorised anchors and transfer ownership. Reads (`verifyRoot`, `isRevoked`, counters) are public, and events are emitted for anchoring, revocation, authorisation and ownership changes. No document data is stored on-chain. The contract is deployed on Polygon Amoy (chain id 80002) [28] at `0x483f9FF4B7444c60e93808Ea0e9b72a14b8Cb12a`, in block 48,502,677 on 25 September 2026, per the repository's deployment record; its source is published on the Amoy block explorer. We could not independently query the chain from the environment used to prepare this paper. Eighteen Hardhat tests cover anchoring, duplicate rejection, zero-value rejection, batch length mismatch, idempotent revocation, authorisation, ownership transfer and views.

VII. PRIVACY AND DATA PROTECTION

A. Envelope Encryption with Fail-Closed Reads

A Prisma query extension seals selected columns before they are written and opens them on read, so services handle plaintext while Postgres holds only envelopes (Table IV). A master key (an environment secret) is expanded with HKDF-SHA256 [29] into a field key-encryption key; each write generates a fresh 256-bit data key, wrapped under that key; and each value is sealed with AES-256-GCM [30] using a fresh 96-bit IV and the field name as associated data, so a ciphertext moved to a differently named field fails authentication. The envelope has the form `cvenc:v1:<keyId>:<wrappedKey>:<iv>:<ciphertext||tag>`. At start-up the cipher round-trips a canary value and refuses to boot if the key store cannot decrypt what it encrypts.

TABLE IV
PROTECTION OF STORED DATA

Data	Protection
Document content, salt, both signatures, revocation reason; version content and change summary	AES-256-GCM envelope (field-bound AAD)
Issued PDFs	Encrypted before storage; deleted on erasure
Organisation private keys	AES-256-GCM, wrapped under the master key
Passwords; refresh and magic-link tokens; API keys	bcrypt (cost 12); SHA-256 digests
Plaintext by design: document hash, signing public key, e-mail, names, embeddings, audit and notification text	None at application level (provider disk encryption only)

In production the server runs with *strict reads*: a non-envelope value in an encrypted column is refused rather than returned as legacy data. This matters because verification trusts the pinned public key stored in the row. A party who can write to the database, but cannot create an envelope without the master key, can only plant plaintext; strict mode refuses it on read, the public lookup then fails closed as INVALID and

discloses nothing, and the batch integrity gate skips the row so the anchor wallet never signs a root containing it.

B. Erasure by Salt Deletion

Because the ledger is immutable, erasure cannot remove an anchored value. Instead, erasing a holder's account deletes, for every document they hold and in one transaction, the content and the salt and every version snapshot; deactivates share links and API keys; removes matches, messages and extracted skills; anonymises the user record; and writes an identifier-only audit entry. Stored PDFs are deleted after commit. What remains is the issuer's record: type, status, dates, revocation information, the hash, both signatures and the proof. Since the salt was 256 bits of randomness, the remaining hash cannot be brute-forced back to content, and the public lookup reports that the holder exercised erasure and discloses nothing else; a credential download for the document returns HTTP 410. This design is meant to support the right to erasure in GDPR Art. 17 [31] and the correction-and-erasure right in s. 12 of India's DPDP Act [32]. We make no claim of legal compliance: whether the residual hash remains personal data is a legal question, treated in the EDPB's consultation draft on blockchains [33], that we have not analysed.

C. Third-Party Egress

When an LLM is configured, skill extraction sends document text to an external API. The text is built from a default-deny allow-list of ten skill-bearing fields (such as designation, department and the free-text assessment fields); compensation, statutory identifiers, names and contact details are never sent, and a pattern-based deny-list backs up the allow-list. Prose fields can name the subject; the team accepts this residual because the prose is the skill signal. Extraction runs only for documents the holder marked for it, and only once issued.

VIII. Consent-Gated, Explainable Talent Matching

A holder opts in per document (skill extraction) and per account (discoverability). After issuance the AI service extracts skills, title, seniority, years of experience and industries, using an LLM (Llama 3.3 70B through an OpenAI-compatible API) when a key is configured and a fixed-vocabulary heuristic otherwise, and embeds the semantic profile into 384 dimensions with all-MiniLM-L6-v2 [20] (with a deterministic hashing fallback). Embeddings are stored in a pgvector column with HNSW indexes. A recruiter creates a job opening; the server embeds it, retrieves the closest consented candidates by cosine distance, keeping each holder's best document and taking the top ten, computes six features (Table V) and sends them to the ranker. Search is scoped to the recruiter's organisation or all organisations, as configured; match results hide holders who have since opted out. The per-holder `DISTINCT ON` query shape scans the filtered set exactly and so does not use the HNSW indexes; the team chose correctness over latency at the current scale.

TABLE V
RANKER FEATURES AND WEIGHTS

Feature	Definition	Weight
embedding_similarity	cosine similarity, clamped to [0,1]	0.40
skill_overlap	share of required skills present	0.25
seniority_fit	1 − Δlevel/3 (0.5 if unknown)	0.15
industry_overlap	share of candidate industries named in the opening	0.10
years_exp_fit	1 if years ≥ minimum, else years/minimum	0.07
recency	1 − age in days/365, clamped	0.03

What the ranker is. The weights in Table V were set by hand. There is no outcome data. To obtain a tree structure over which TreeSHAP can attribute, the service fits a LightGBM regressor (200 trees, 31 leaves) on 2,000 synthetic samples drawn uniformly at random, whose labels are the weighted sum of the features plus small noise. The booster therefore re-derives, to within fitting error, the hand-set weighted sum. It adds no real-world signal. Attributions are computed with LightGBM's native TreeSHAP and reconcile with the score (base value plus contributions); the model version string is deliberately self-describing (`heuristic-lgbm-approx`) so that no consumer mistakes it for a trained model, and a plain weighted sum is used if LightGBM is unavailable. The explanation panel therefore faithfully explains *this scoring function*; it is not evidence of what predicts a good hire. The intended label source, recruiter-confirmed positives from the interested/not-interested responses to outreach, exists in the schema but has not been used for training.

IX. Evaluation

A. Cost of the Cryptographic Primitives

We timed the primitives with the same libraries the server uses (`canonicalize` for RFC 8785, `merkletreejs` with SHA-256 and sorted pairs, and Node's `crypto` for SHA-256 and RSA-2048/RS256) on an 11th-gen Intel Core i5-1135G7 at 2.4 GHz under Node 26.10, using a representative 10-field experience letter. These are micro-benchmarks of single operations, not end-to-end latency: they exclude the database, encryption, networking and chain confirmation, and no load test was run.

TABLE VI
MEASURED COST OF PRIMITIVE OPERATIONS (MEAN)

Operation	Time
JCS + salt + SHA-256 (document hash)	5.4 µs
Role-statement hash	4.5 µs
RS256 sign (RSA-2048)	271 µs
RS256 verify	20.6 µs
Merkle proof verification, n = 1,000	27.7 µs
Merkle tree construction, n = 100,000	278 ms

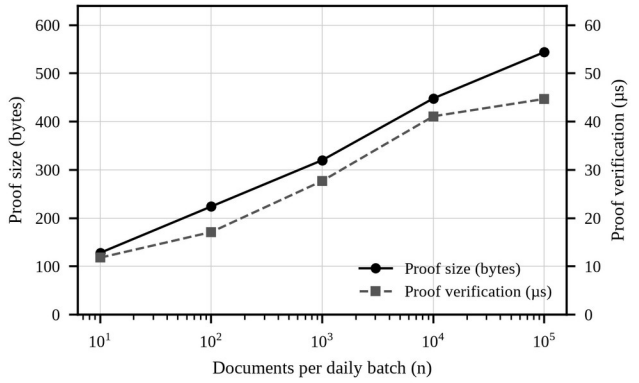


Fig. 4. Merkle proof size and verification time versus daily batch size n (measured).

Issuing a document needs two signatures, about 0.54 ms of signing time, and verifying both takes about 41 µs. Proof size grows as $\lceil \log_2 n \rceil$ (Eq. 5): 128 bytes for ten documents and 544 bytes for 100,000, while verification stays under 50 µs (Fig. 4). Building a tree over 100,000 leaves takes 278 ms. Because one `anchorRoot` transaction covers a whole batch, the on-chain cost per document falls as $1/n$ (R8). We did not measure gas; the registry stores one record per root, and the project's own documentation reports roughly 137,500 gas per anchor, which we have not reproduced.

B. Test Inventory

Table VII counts test cases statically. In the environment used to prepare this paper, which lacked some optional dependencies, 22 of the 31 server unit-test suites could be executed and all 226 of their tests passed, and all 17 AI-service tests passed. The remaining server suites, the integration and browser suites and the contract tests were not re-executed here.

TABLE VII
TEST INVENTORY (STATIC COUNT)

Suite	Scope	Cases
Server unit (31 spec files)	crypto, Merkle, gate, encryption, verification, RBAC	≈275
Server integration (5 files)	encryption, strict reads, erasure, signing key	26
Playwright (5 files)	org, lifecycle, bulk issuance, RBAC, sessions	25
Contract (Hardhat)	AnchorRegistry behaviour	18
AI service (pytest)	extraction, embeddings, ranking, config, API	17
Verifier vectors	JCS, hash, statements, Merkle (known answers)	—

C. Threat Analysis

Table VIII walks through the adversaries of Section III. The first column is an attempted action; the second is the outcome in the current implementation; the third names the mechanism.

TABLE VIII
ATTEMPTED ATTACKS AND OUTCOMES

Attempt	Outcome	Mechanism
A1 edits content after issuance	Detected; INVALID	Hash recomputed from content and salt
A1 reuses manager signature as HR's	Detected	Role and member id inside signed statement
A2 reads a dump or SQL console	Sealed fields unreadable; hash, e-mail, names visible	AES-256-GCM envelopes (Table IV)
A3 plants a self-consistent row with own key	Refused on read; not anchored (production)	Strict reads; batch integrity gate
A3 swaps a document's pinned key	Signatures fail: denial of service, no forgery	Signature check against pinned key
A3 grants itself manager and HR roles, then issues via the app	**Succeeds**	Membership rows are plaintext and unaudited
A4 holds the master key	Can sign as any organisation; cannot alter past anchors or downloaded credentials	Chain is append-only; offline proof
A5 issues a false but well-formed letter	Verifies; truth is not established	Out of scope (Section III)
Forged credential with attacker's key	Offline checks pass; key fingerprint differs	Out-of-band fingerprint check
A6 chain unreachable	PENDING, not INVALID	Local proof check; 10 s RPC bound
Operator offline	Offline verification still works	Self-contained credential file

X. LIMITATIONS

We list limitations of the current implementation without softening them.

- **One custodial key per organisation.** Both signatures come from one key held by the platform. Separation of duties rests on the application, and a compromised operator can sign as any organisation.
- **Master key as an environment secret.** There is no HSM or cloud KMS and no master-key rotation tooling; only local-disk and Supabase-bucket key stores exist.
- **Domain verification is bypassed in the demo deployment.** With the local DNS driver every domain passes, so in that deployment issuer authenticity (R1) is not established. The real TXT-record driver is implemented but not enabled there.
- **Mocked payments and e-mail.** Billing, share-link checkout and subscriptions use a mock provider; e-mail is written to the console in the demo.
- **Testnet only.** Anchors sit on Polygon Amoy, a test network without the economic security of a production chain. The anchor wallet is a single hot key that is also the registry owner.
- **Revocation is database-authoritative.** A verifier must trust the operator's database for revocation; the on-chain flag is written without retry and can be missing.
- **The ranker is not trained.** Section VIII states what it is. No fairness, accuracy or user study has been run on the matching feature.
- **Plaintext identifiers.** E-mail addresses and names are stored in clear for login lookup; membership grants are

neither signed nor audited, and the associated data of an envelope binds the field but not the row.

- **Merkle construction.** Merkle leaves are raw document hashes and are not domain-separated from interior nodes as in [12]; this is not exploitable here because a verifier recomputes a leaf from content and salt, but it is not a hardened construction.
- **Scale and assurance.** Rate limits and schedulers are in-process and single-instance; no external security audit, formal verification or load test has been performed.

XI. Planned Work

The project's roadmap lists the following items as planned, none of them implemented: an external KMS or HSM for organisation keys and per-member signing keys; deployment of the registry on a production Polygon network with a multisig owner; mirroring of Merkle roots and proofs to IPFS and a public repository; real payment metering, cloud object storage and transactional e-mail; a blind index for e-mail and names; auditing or signing of membership grants and a key-history check for pinned keys; binding envelope associated data to the row; pinning the database CA; and distributed rate limiting and job queues.

XII. Conclusion

CareerVault shows that employment documents can be issued with role-bound dual approval, verified in seconds by an anonymous party, and re-verified offline from a single file, while only a 32-byte Merkle root per day touches a public ledger and holders retain a path to erasure. Measured primitive costs are small: proofs stay below 600 bytes for 100,000 documents and verify in tens of microseconds. The system's guarantees are conditional, and we have stated the conditions: one custodial key per organisation, a database that remains authoritative for revocation, a demo deployment with domain checks bypassed, and a ranker that explains a hand-set scoring function rather than a learned one. Closing those gaps is the work that separates this implementation from a production trust service.

References

[1] "AuthBridge exposes India's hiring red flags," *CXOToday*, 2025. [Online]. Available: https://cxotoday.com/press-release/authbridge-exposes-indias-hiring-red-flags/

[2] "HireRight's latest study reveals resume fraud in India," *People Matters*, 2015. [Online]. Available: https://www.peoplematters.in/article/background-verification/hirerights-latest-study-reveals-resume-fraud-india-11343

[3] V. Marella and A. Vijayan, "Document verification using blockchain for trusted CV information," in *Proc. Americas Conf. Inf. Syst. (AMCIS)*, 2020, Art. 12. [Online]. Available: https://aisel.aisnet.org/amcis2020/adv_info_systems_research/adv_info_systems_research/12

[4] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," *J. Cryptology*, vol. 3, no. 2, pp. 99–111, 1991.

[5] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology—CRYPTO '87*, ser. LNCS, vol. 293. Berlin, Germany: Springer, 1988, pp. 369–378.

[6] "OpenTimestamps," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/OpenTimestamps

[7] "MIT pilots digital diplomas based on Bitcoin's blockchain," *Campus Technology*, Oct. 19, 2017. [Online]. Available: https://campustechnology.com/Articles/2017/10/19/MIT-Pilots-Digital-Diplomas-Based-on-Bitcoins-Blockchain.aspx

[8] M. Turkanović, M. Hölbl, K. Košič, M. Heričko, and A. Kamišalić, "EduCTX: A blockchain-based higher education credit platform," *IEEE Access*, vol. 6, pp. 5112–5127, 2018, doi: 10.1109/ACCESS.2018.2789929.

[9] W. Gräther *et al.*, "Blockchain for education: Lifelong learning passport," in *Proc. 1st ERCIM Blockchain Workshop*, EUSSET, 2018, doi: 10.18420/blockchain2018_07.

[10] P. Herbke, A. Sapkota, and S. Lamichhane, "Lifecycle management of resumés with decentralized identifiers and verifiable credentials," arXiv:2406.11535, 2024.

[11] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *J. Manage. Inf. Syst.*, vol. 24, no. 3, pp. 45–77, 2007, doi: 10.2753/MIS0742-1222240302.

[12] B. Laurie, E. Messeri, and R. Stradling, "Certificate Transparency version 2.0," IETF, RFC 9162, Dec. 2021.

[13] E. Androulaki *et al.*, "Hyperledger Fabric: A distributed operating system for permissioned blockchains," in *Proc. 13th EuroSys Conf.*, Porto, Portugal, 2018, doi: 10.1145/3190508.3190538.

[14] M. Baldi, F. Chiaraluce, M. Kodra, and L. Spalazzi, "Security analysis of a blockchain-based protocol for the certification of academic credentials," arXiv:1910.04622, 2019.

[15] M. Sporny, T. Thibodeau Jr., I. Herman, G. Cohen, and M. B. Jones, Eds., "Verifiable Credentials data model v2.0," W3C Recommendation, May 15, 2025. [Online]. Available: https://www.w3.org/TR/vc-data-model-2.0/

[16] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017.

[17] S. M. Lundberg *et al.*, "From local explanations to global understanding with explainable AI for trees," *Nat. Mach. Intell.*, vol. 2, no. 1, pp. 56–67, Jan. 2020, doi: 10.1038/s42256-019-0138-9.

[18] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017.

[19] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, Hong Kong, 2019, pp. 3982–3992, doi: 10.18653/v1/D19-1410.

[20] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.

[21] "pgvector: Open-source vector similarity search for Postgres." [Online]. Available: https://github.com/pgvector/pgvector

[22] Yu. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020, doi: 10.1109/TPAMI.2018.2889473.

[23] Regulation (EU) 2024/1689 of the European Parliament and of the Council laying down harmonised rules on artificial intelligence (Artificial Intelligence Act), Annex III, *Off. J. Eur. Union*, 2024.

[24] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf.*, Monterey, CA, USA, 1999, pp. 81–92.

[25] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015.

[26] A. Rundgren, B. Jordan, and S. Erdtman, "JSON Canonicalization Scheme (JCS)," IETF, RFC 8785, Jun. 2020.

[27] K. Moriarty, Ed., B. Kaliski, J. Jonsson, and A. Rusch, "PKCS #1: RSA cryptography specifications version 2.2," IETF, RFC 8017, Nov. 2016.

[28] “Polygon PoS: RPC endpoints,” Polygon Docs. [Online]. Available: https://docs.polygon.technology/pos/reference/rpc-endpoints/

[29] H. Krawczyk and P. Eronen, “HMAC-based extract-and-expand key derivation function (HKDF),” IETF, RFC 5869, May 2010.

[30] M. Dworkin, “Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC,” NIST, Gaithersburg, MD, USA, SP 800-38D, Nov. 2007.

[31] Regulation (EU) 2016/679 of the European Parliament and of the Council (General Data Protection Regulation), art. 17, *Off. J. Eur. Union*, L 119, May 4, 2016.

[32] Digital Personal Data Protection Act, 2023, s. 12, Govt. of India, 2023.

[33] European Data Protection Board, “Guidelines 02/2025 on processing of personal data through blockchain technologies,” version for public consultation, Apr. 2025.